

Toy Clinical Trial via Google Sheets

Active vs. Placebo Memory Enhancement Study, authored in Sheets and imported into ZZedc

ZZedc Team

2026-04-28

Overview

This vignette is the Google-Sheets-driven companion to the CSV-based Toy Clinical Trial Setup walk-through. The study is the same — *Active vs. Placebo Memory Enhancement Study with 20 subjects* — but the data dictionary, validation rules, and user roster are authored in Google Sheets rather than in CSV files. A single import call materialises them into the ZZedc database.

This path is intended for clinical teams that prefer a familiar spreadsheet interface for ongoing CRF and validation-rule maintenance:

- Study coordinators without R fluency can edit forms and rules directly in a browser
- Multi-author edits and revision history are handled by Google Sheets natively
- Bidirectional sync allows Sheets and the ZZedc database to stay aligned through the life of the trial

The CSV path remains available and is preferred for studies that will live entirely on a single laptop, in air-gapped environments, or anywhere Google Workspace is not an option.

Prerequisites

ZZedc must be installed and at least the SQLite backend configured. Follow Getting Started first if you have not done so.

Install the optional Google Sheets dependency

`googlesheets4` is in `Suggests`, not `Imports`. Install it explicitly the first time you take this path:

```
install.packages("googlesheets4")
```

Choose an authentication mode

ZZedc supports both authentication modes that `googlesheets4` provides:

- **Interactive OAuth** — recommended for development and small teams. The first call opens a browser, prompts for consent, and caches a refresh token at `~/.cache/gargle/`.
- **Service account** — required for unattended jobs (CI, cron, containerised deployments). Create a service-account key in Google Cloud Console, share the target Sheet with the service account's email address, then point `googlesheets4` at the JSON key.

```
library(googlesheets4)

# Interactive mode (one-time browser consent):
gs4_auth(email = "your-google-account@example.org")
```

```
# Service-account mode (unattended):
gs4_auth(
  path = "/path/to/zzedc-service-account.json",
  scopes = "https://www.googleapis.com/auth/spreadsheets"
)
```

Trial structure (recap)

Identical to the CSV vignette:

Timeline:

Visit 1 (Baseline) -> Randomization -> Visit 2 (Week 4) -> Visit 3 (Week 8)

Forms:

1. Enrollment (subject_id, enrollment_date, age, gender, education_years, medical_history)
2. Randomization (subject_id, randomization_date, treatment_assignment, study_coordinator)
3. MMSE (subject_id, visit_label, assessment_date, 7 component scores, mmse_total, adverse_events, ae_description)

Population:

- 20 subjects, 1:1 randomization (10 active / 10 placebo)

Sheet structure

A single Google Workbook with five tabs holds everything the toy trial needs. The tab names and column schemas below match what ZZedc's importers expect.

Tab 1: Study_Users

Authentication / role roster. Plain-text passwords are hashed by ZZedc on import; never store production credentials in a Sheet intended to be shared.

Column	Required	Example
username	yes	jane_smith
full_name	yes	Jane Smith
email	yes	jane@example.org
role	yes	Coordinator
password_initial	yes	jane123
site_id	no	001
active	no	TRUE

Roles must match values returned by `zzedc::list_user_roles()` (Admin, PI, Coordinator, Researcher, DataManager, Monitor in the default schema).

Tab 2: Data_Dictionary

CRF field definitions, one row per field across all forms. The column shape mirrors the CSV templates in the CSV vignette:

Column	Required	Example
form_code	yes	<code>mmse</code>
field_name	yes	<code>attention</code>
field_label	yes	Attention and Calculation
field_type	yes	numeric
required	yes	TRUE
validation	no	<code>between 0 and 5</code>
description	no	Score 0-5 points
field_order	no	4

Tab 3: validation_rules

DSL rules in plain-English syntax. The schema matches what `import_validation_rules_from_gsheets()` expects (see `?import_validation_rules_from_gsheets` for the full column list).

Column	Required	Example
rule_id	yes	<code>MMSE_TOTAL_RANGE</code>
field_code	yes	<code>mmse_total</code>
rule_dsl	yes	<code>mmse_total between 0 and 30</code>
form_code	no	<code>mmse</code>
rule_name	no	MMSE total in valid range
error_message	no	MMSE total must be 0-30
severity	no	ERROR (default)
rule_category	no	FIELD (default)
is_active	no	TRUE (default)

A small starter set for the toy trial:

rule_id	field_code	rule_dsl
AGE_RANGE	age	<code>age between 50 and 90</code>
MMSE_TOTAL	mmse_total	<code>mmse_total between 0 and 30</code>
MMSE_SUM_CONSISTENT	mmse_total	<code>mmse_total = orientation_time + orientation_place + registration + attention + recall + language + visual_spatial</code>
RAND_AFTER_ENROLLMENT	randomization_date	<code>randomization_date >= enrollment_date</code>
VISIT_WINDOW	assessment_date	<code>assessment_date - randomization_date between -1 and 70</code>

Tab 4: Visit_Schedule (optional)

Visit windows and form assignments per visit. ZZedc derives defaults if absent.

visit_code	visit_name	nominal_day	window_minus	window_plus	forms
BASELINE	Baseline	0	0	0	enrollment,mmse
WK4	Week 4	28	3	3	mmse
WK8	Week 8	56	3	7	mmse

Tab 5: Subjects (optional, for seeded data)

Used only if you want to pre-populate subject records, as the toy trial does. In a real study this tab is omitted; subjects are entered through the EDC interface as they enrol.

Identical schema to `subjects.csv` in the CSV vignette.

Generating a template

ZZedc provides a helper that creates a Google Sheet pre-populated with the validation-rules schema and example rows:

```
library(zzedc)

template <- create_validation_rules_template(
  sheet_name      = "ToyTrial_ValidationRules",
  include_examples = TRUE
)
template$sheet_url
```

The helper covers the `validation_rules` tab specifically. For the `Study_Users` and `Data_Dictionary` tabs, use the column lists in the section above as a manual template, or copy from the CSV files in `vignettes/toy-trial/csv_templates/` and paste into your Sheet.

Importing a populated workbook

Once the workbook has the four required tabs filled in (`Study_Users`, `Data_Dictionary`, `validation_rules`, optionally `Visit_Schedule` and `Subjects`), import it into a fresh ZZedc database with a single call:

```
library(zzedc)

db_path <- "~/zzedc_toy_trial_gsheets/data/toy_trial.db"
dir.create(dirname(db_path), recursive = TRUE,
            showWarnings = FALSE)

sheet_id <- "1AbC...your-sheet-id..." # from the Sheet URL

result <- setup_zzedc_from_gsheets(
  sheet_id      = sheet_id,
  db_path       = db_path,
  imported_by   = "data_manager",
  overwrite     = TRUE
)

# Per-stage status
result$success           # overall pass/fail
result$users$imported    # number of users imported
result$forms$imported_forms # number of CRFs created
result$forms$imported_fields # number of fields across all CRFs
result$validation_rules$imported # number of validation rules imported

# The encryption key is needed for any subsequent operations in
```

```
# this R session.
Sys.setenv(DB_ENCRYPTION_KEY = result$key)
```

setup_zzedc_from_gsheets() runs four stages in order:

1. Initialise the encrypted SQLite database at db_path.
2. Read Study_Users and create user accounts.
3. Read Data_Dictionary, create one CRF per form_code value, and add each field as defined.
4. Read validation_rules and import the DSL rules through [import_validation_rules_from_gsheets()] (which itself validates DSL syntax row-by-row).

If any stage fails, the others still run and the failures are collected in stage-specific **errors** lists; the top-level **success** flag is TRUE only if every stage passed cleanly.

Validating before importing

Use `dry_run = TRUE` to parse and validate the workbook without writing to the database. The dry-run pass exercises the DSL parser on every validation rule and reports counts the same way:

```
preview <- setup_zzedc_from_gsheets(
  sheet_id = sheet_id,
  db_path  = db_path,
  dry_run  = TRUE
)
preview$validation_rules$skipped # rule rows with syntax errors
preview$validation_rules$errors  # named list of error messages
```

Importing only the validation rules

If you already have an initialised database and only need to refresh the validation-rule tab, call the lower-level importer directly:

```
result <- import_validation_rules_from_gsheets(
  sheet_id      = sheet_id,
  sheet_name    = "validation_rules",
  imported_by   = "data_manager",
  validate_syntax = TRUE,
  dry_run       = FALSE
)
```

This is the underlying call `setup_zzedc_from_gsheets()` makes in its fourth stage.

Bidirectional sync

For studies that maintain CRFs and validation rules in Google Sheets as the source of truth, ZZedc supports periodic re-import:

```
sync_result <- sync_dsl_rules_from_gsheets(
  sheet_id      = sheet_id,
  imported_by   = "data_manager",
  delete_missing = FALSE
)

sync_result$added
sync_result$updated
sync_result$deleted
```

`delete_missing = FALSE` (default) treats Sheet rows as additive: rules absent from the Sheet are left alone in the database. Set to `TRUE` when the Sheet is the canonical authority and missing rules should be retired.

For audit purposes, every sync run writes a record to the `audit_trail` table tagged with the `sheet_id`, the timestamp, and the user who ran it.

Verification

```
conn <- connect_encrypted_db(db_path = db_path,
                             key = init_result$key)
on.exit(DBI::dbDisconnect(conn), add = TRUE)

# Roster
DBI::dbGetQuery(conn, "SELECT username, role FROM edc_users")

# Form / field counts
DBI::dbGetQuery(conn, "
  SELECT crf_code, COUNT(*) AS n_fields
  FROM crf_definitions cd
  JOIN designer_fields df ON df.design_id = cd.crf_id
  GROUP BY crf_code
")

# Validation rules
DBI::dbGetQuery(conn, "
  SELECT rule_id, field_code, severity FROM validation_rules
")
```

Launching the application

Once the database is populated, the launch path is identical to the CSV vignette:

```
library(zzedc)
Sys.setenv(ZZEDC_DB_PATH = db_path)
launch_zzedc()
```

The Shiny session opens in your default browser. Authenticate as one of the users you imported and proceed to data entry. The audit log records every action, the validation rules fire as defined in the Sheet, and exports are available through the secure-export module.

Operating model

In practice, the Google-Sheets path is most useful when:

- The data manager and study coordinator share the Sheet; the data manager owns validation rules and the coordinator owns CRF text and the user roster.
- A non-trivial CRF revision is anticipated mid-study; the Sheet provides the revision history and conflict-resolution surface Google Workspace already maintains.
- The deployment is multi-site and at least one site cannot run R locally; that site can still author CRFs in the Sheet, and the central data manager imports.

For a strictly single-PI laptop deployment, the CSV path (`vignette('toy-trial-setup')`) is simpler.

Limitations

The Google-Sheets path has reached parity with the CSV path for study setup, but a few constraints remain:

- Bidirectional sync currently handles the validation-rules tab only; users and form definitions are import-only. To revise CRFs or the user roster after initial setup, edit the database directly through the ZZedc admin interface or rerun `setup_zzedc_from_gsheets()` against a fresh `db_path`.
 - Service-account quotas apply: at the time of writing, `googlesheets4` defaults to the standard Google Sheets API rate limits (300 read requests per minute per user). Large deployments should batch reads via `range_read()` rather than per-tab calls.
 - Stage-level errors surface in the result object's per-stage `errors` lists rather than aborting the import. Inspect `result$success` and the per-stage objects before assuming a successful import.
-

Troubleshooting

Authentication fails with `gs4_auth()`: clear the cached token (`gargle::gargle_oauth_cache()` shows the location) and re-authenticate. If running unattended, switch to a service account.

Missing required columns: the importer requires `rule_id`, `field_code`, `rule_dsl` for the validation tab. Match column names exactly (case-sensitive).

Syntax error in DSL: rerun the import with `dry_run = TRUE` and inspect `result$errors` to identify the offending row. The DSL parser accepts plain-English forms like `age between 18 and 120`, `enrollment_date <= randomization_date`, `mmse_total = orientation_time + ....`. See the validation-rules section of Advanced Features for the full grammar.

Sheet not found: verify the Sheet has been shared with the service account email (when using a service account) or with the user account that authenticated (when using interactive OAuth).

Resources

- Toy Clinical Trial Setup (CSV path)
- Advanced Features: Google Sheets Integration
- `?setup_zzedc_from_gsheets`
- `?import_validation_rules_from_gsheets`
- `?sync_dsl_rules_from_gsheets`
- `?create_validation_rules_template`
- `googlesheets4` documentation